



计算机图形学与虚拟现实

冼楚华

Email: chhxian@scut.edu.cn

华南理工大学计算机科学与工程学院



课程信息

- 授课老师姓名：冼楚华
- Email: chhxian@scut.edu.cn 
- 个人主页: <https://chuhuaxian.github.io/>
- QQ: 89071086 （比较少用，非急事请不要私聊）
- 办公室: B3-202-2
- 课程QQ群（见二维码）



群名称: 2025计算机图形学@SCUT
群 号: 134660465

Agenda

- 消隐(Hidden surface removal, HSR)
 - 基本概念(Terminologies)
 - 图像空间算法(Image space)
 - z缓冲算法(Z-buffer algorithm)
 - 对象空间算法(Object space)
 - 后向面剔除(back surface culling)
 - 表优先级算法(list priority methods)
 - 深度排序算法/画家算法/
 - 二叉划分树算法

Agenda

- 消隐(Hidden surface removal, HSR)
 - 基本概念(Terminologies)
 - 图像空间算法(Image space)
 - z缓冲算法(Z-buffer algorithm)
 - 对象空间算法(Object space)
 - 后向面剔除(back surface culling)
 - 表优先级算法(list priority methods)
 - 深度排序算法/画家算法/
 - 二叉划分树算法

1 什么是消隐？

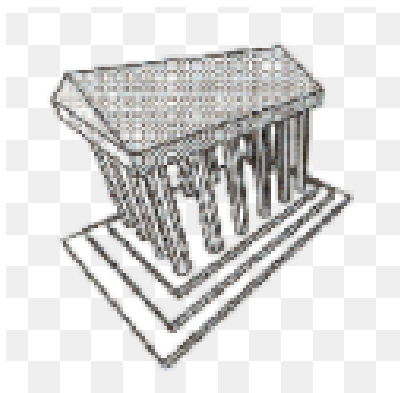
- 消隐——消除隐藏面 (Hidden Surface Removal):

- 从几何场景中删除不在视域体内的几何图元(裁剪)
- 删除视域中点不可见或被其他几何体遮挡的物体(消隐)
- 具体实现：处理多边形面

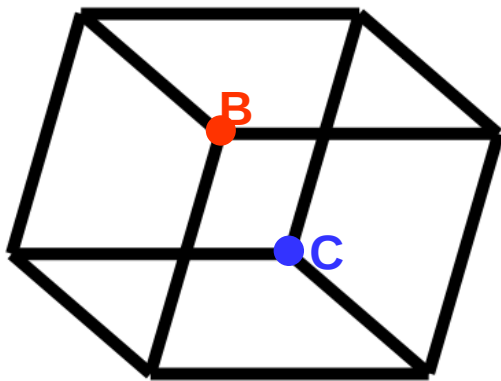


为什么要消隐？

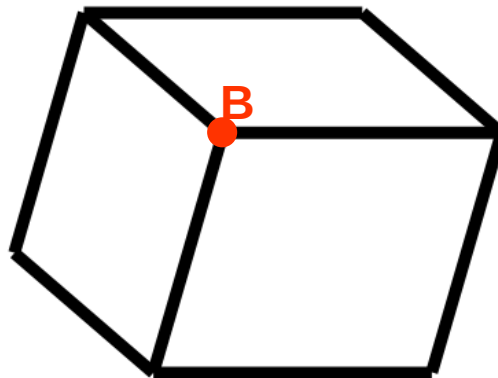
- 图形真实感方法(Enhance photorealistic)
 - 透视投影 (Projection: 3D space $\rightarrow$ 2D space)
 - 按深度排序增加3D线索
 - 真实感光照 (Photorealistic illumination)



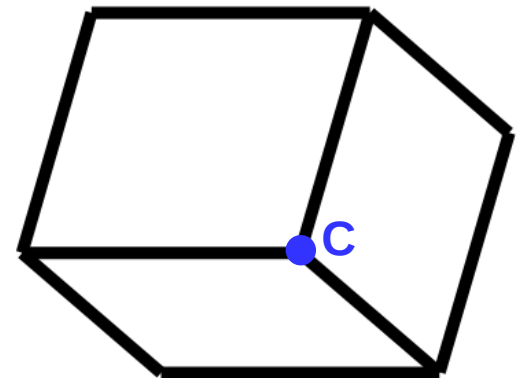
消隐可减少二义性(reduce ambiguity)



(a)



(b)



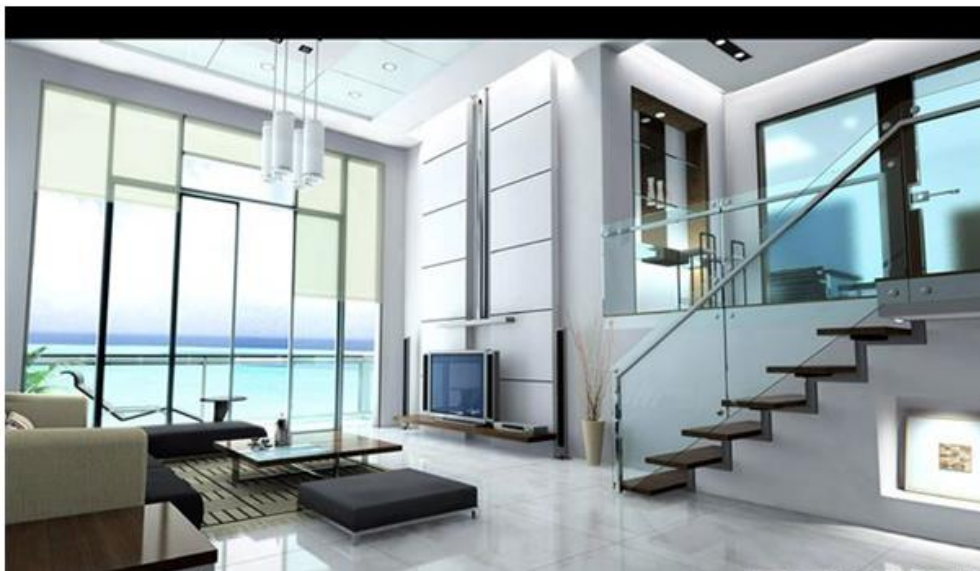
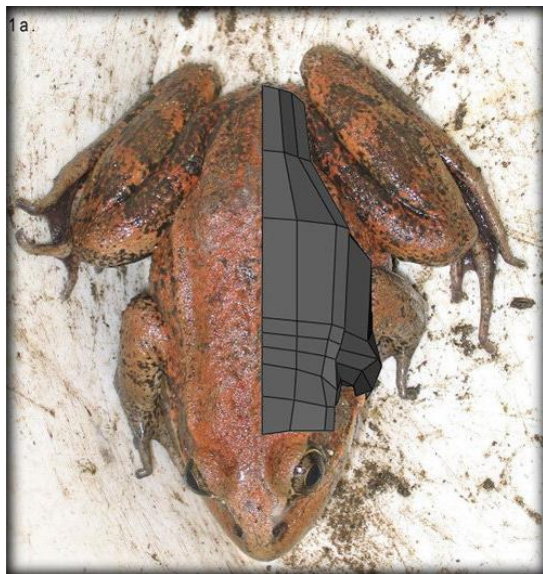
(c)

(a) Cube wireframe; (b) B is the nearest; (c) C is the nearest

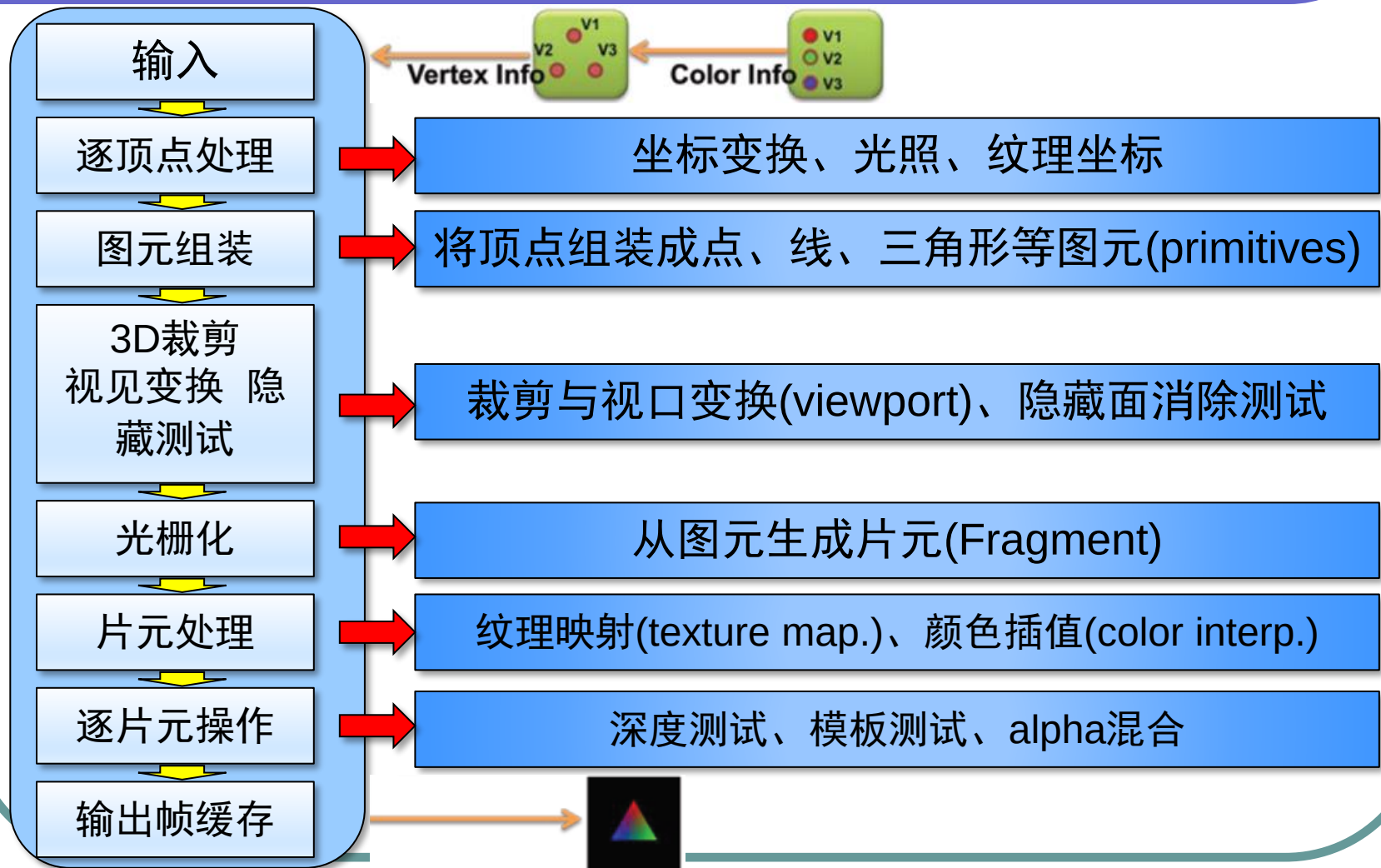
消隐可提高绘制效率

- 消隐可提高绘制效率

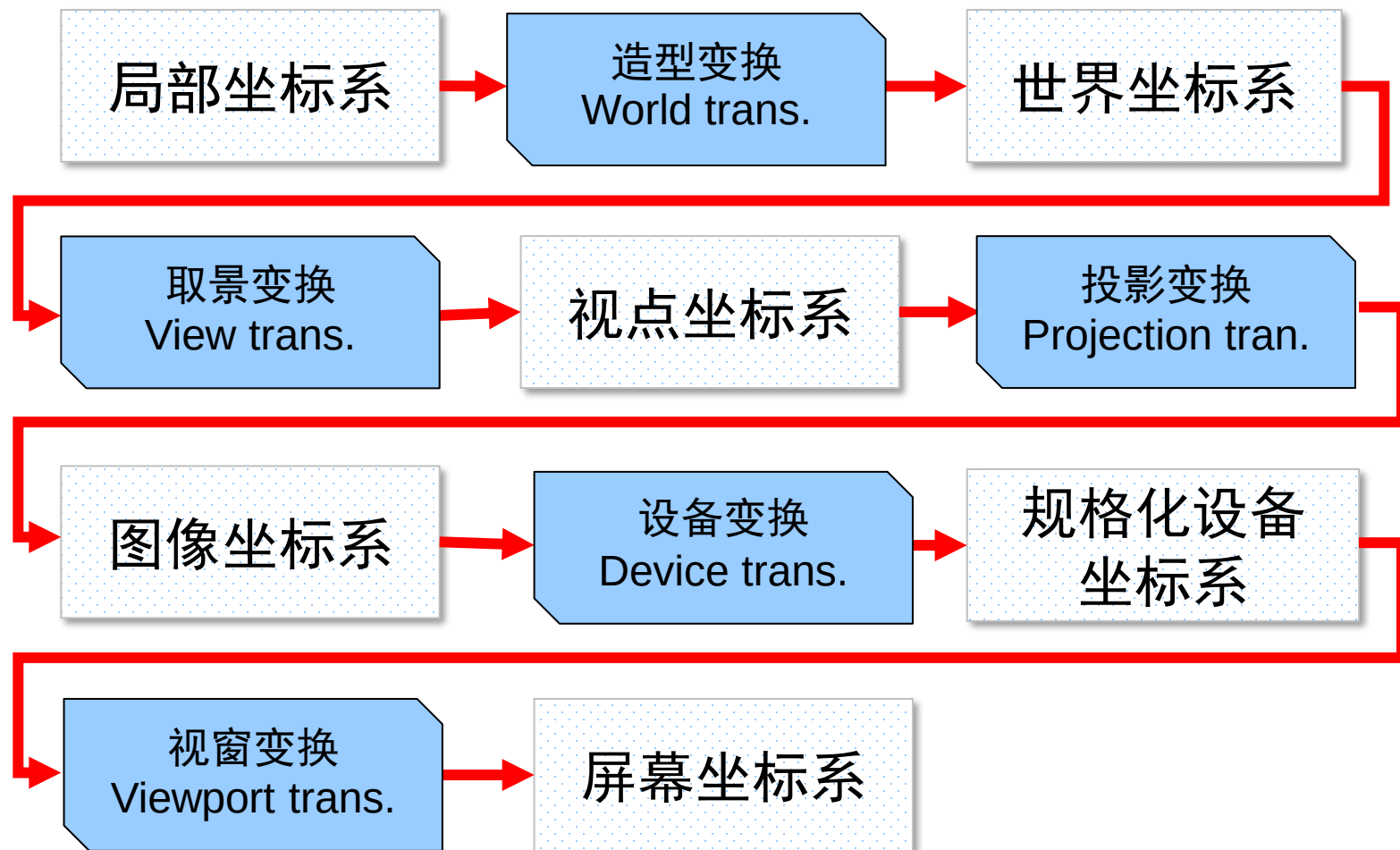
- 实时仿真(realtime simulation): 效率优先
- 真实感绘制(photorealistic rendering): 质量优先



什么时候进行消隐?图形管线(graphics pipeline)

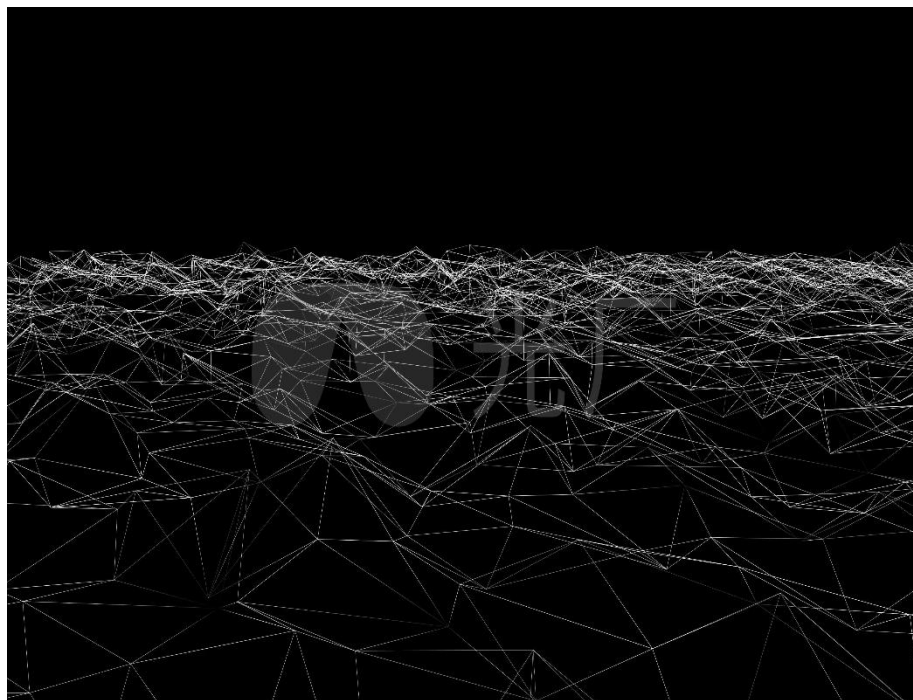


图形处理流程中的坐标变换



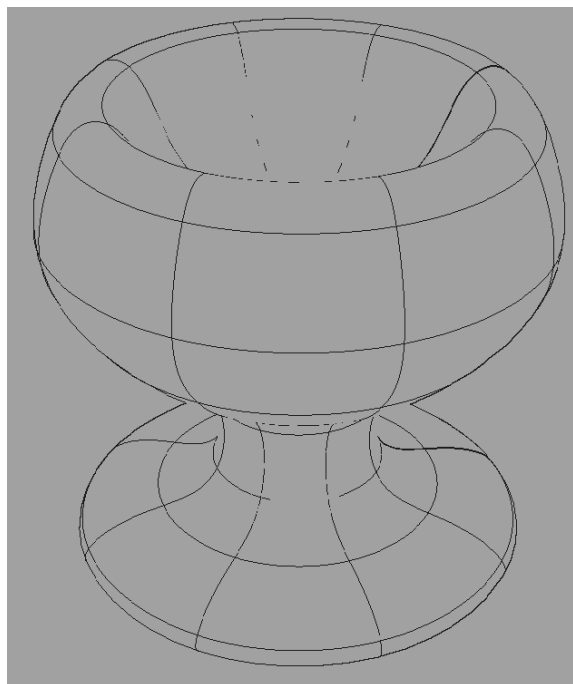
排序与连贯性(HSR:sorting & coherence)

- **排序(Object sorting)**
 - 对场景中物体按其到视点的远近排序.
- **连贯性(Coherence)**
 - 同一区域像素往往有相似性质(可见或遮挡)
- **两者决定了消隐效率**

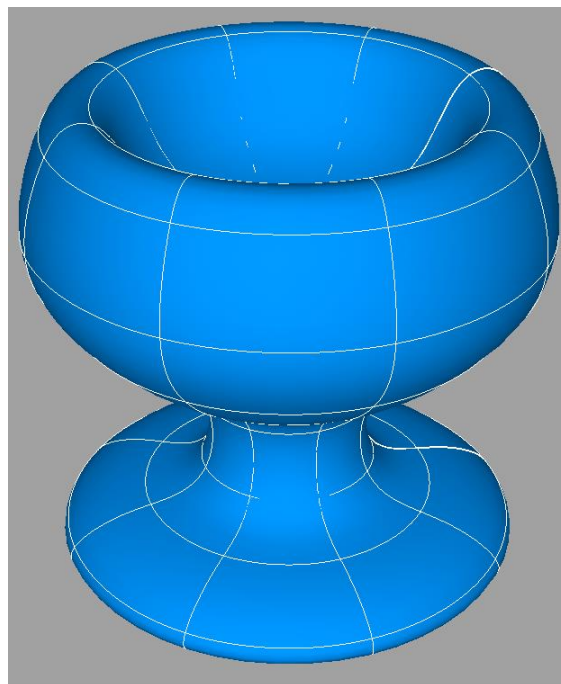


消隐算法分类—按输出形式

- 隐藏线/面消除Hidden line/surface



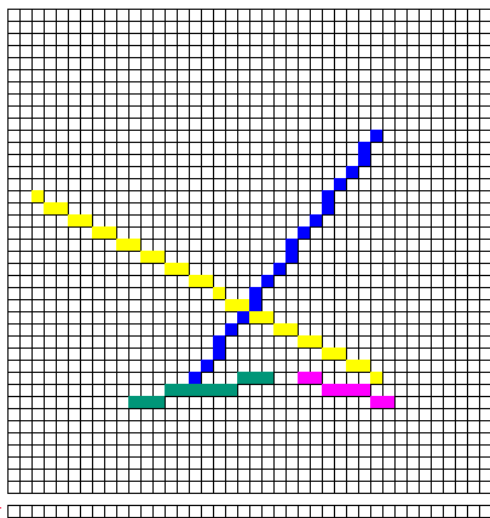
线消隐：输出线框图



面消隐：输出着色图

消隐算法分类—按消隐空间

- 图像空间消隐 (Hidden surface removal in image space)
 - 为显示窗口中的每个像素找到场景中对应的可见点
- 景物空间消隐/对象空间消隐 (Hidden surface removal in object space)
 - 对场景中的多边形按由远到近排序

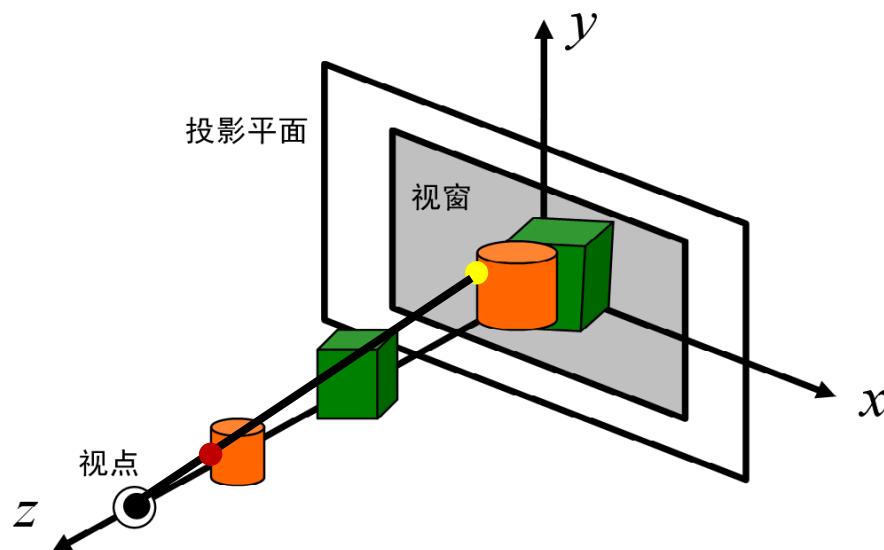


消隐算法(HSR, Textbook: chap 8.11)

- 基本概念
- 图像空间消隐
 - Z-buffer algorithm (深度缓存)
- 对象空间消隐
 - 背向面剔除
 - 表优先级算法(list priority methods)
 - 深度排序算法/画家算法/
 - 二叉划分树算法

2 图像空间消隐---算法框架(HSR in image space)

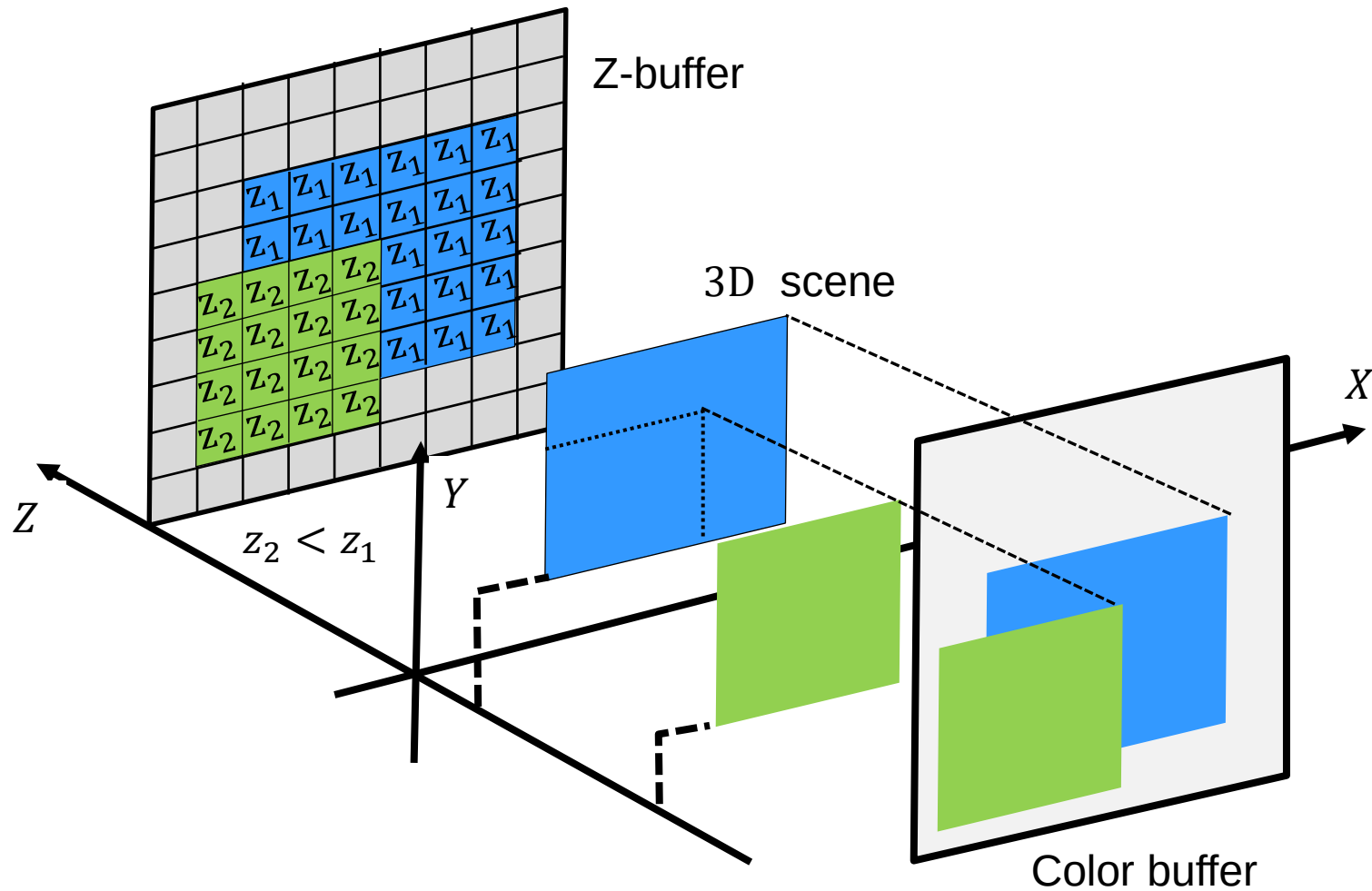
1. for (图像中的每个像素 p) {
2. 连接视点与像素射线 r
3. **求出 r 与场景的最近交点 s**
4. 计算交点 s (像素 p) 颜色
- 5 } }



图像空间算法特点

- 受分辨率限制
 - 生成新的图像需重新消隐
- 复杂度(Complexity): $O(nN)$
 - 每个象素都需对物体排序 (Objects should be sorted for each pixel)
 - n : 多边形个数 (the number of polygons)
 - N : 像素数 (the number of pixels)
- 算法举例
 - Z缓存 (Z-buffer), A缓存 (A-buffer), 扫描线算法 (scan line algorithms)

Z-buffer消隐算法



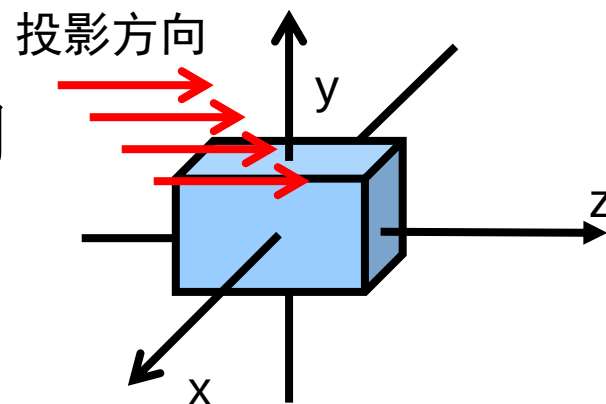
Z-buffer消隐算法

- 帧缓存(Frame-buffer)

- 存储空间与显示窗口像素数相同
- 存像素颜色,也叫颜色缓存

- Z缓存(Z-buffer)

- 存储空间与帧缓存类似
- 存像平面到像素对应场景的距离,称为z值
- Z值越大离视点越远



例: 一个简单场景的z-buffer



Z 值越大离视点越远

1	1	1	1	1	1	1	1	背景深度缓存
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	
1	1	1	1	1	1	1	1	

.5	.5	.5	.5	.5	.5	.5	第1个三角形深度缓存
.5	.5	.5	.5	.5	.5		
.5	.5	.5	.5	.5			
.5	.5	.5	.5				
.5	.5	.5					
.5	.5						
.5							
.5							

.5	.5	.5	.5	.5	.5	.5	1
.5	.5	.5	.5	.5	.5	1	1
.5	.5	.5	.5	.5	1	1	1
.5	.5	.5	.5	1	1	1	1
.5	.5	.5	1	1	1	1	1
.5	.5	1	1	1	1	1	1
.5	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1

.7							
.6	.7						
.5	.6	.7					
.4	.5	.6	.7				
.3	.4	.5	.6	.7			
.2	.3	.4	.5	.6	.7		

第2个三角形深度缓存

.5	.5	.5	.5	.5	.5	.5	1
.5	.5	.5	.5	.5	.5	1	1
.5	.5	.5	.5	.5	1	1	1
.5	.5	.5	.5	1	1	1	1
.4	.5	.5	.7	1	1	1	1
.3	.4	.5	.6	.7	1	1	1
.2	.3	.4	.5	.6	.7	1	1
1	1	1	1	1	1	1	1

z-buffer算法流程

(1) 初始化颜色缓存 $fbuffer$

← 背景色

(2) 初始化深度缓存 $zbuffer$

← 视点最远的 z 值

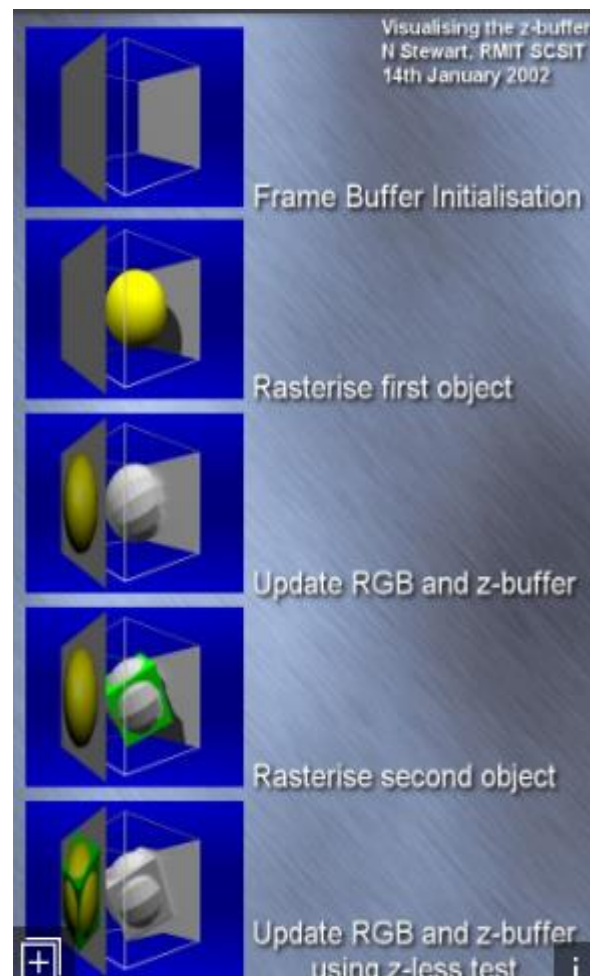
(3) 以任意顺序**遍历并扫描转换**所有多边形

a) $z(x, y) \leftarrow \text{ComputeDepth}(x, y)$

b) 如果 $z(x, y) < zbuffer(x, y)$, 那么

$fbuffer \leftarrow \text{ComputeColor}(x, y)$

$zbuffer(x, y) \leftarrow z(x, y)$



Z-buffer伪代码

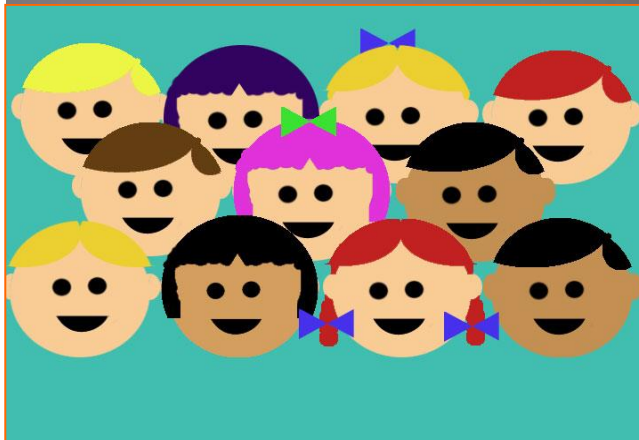
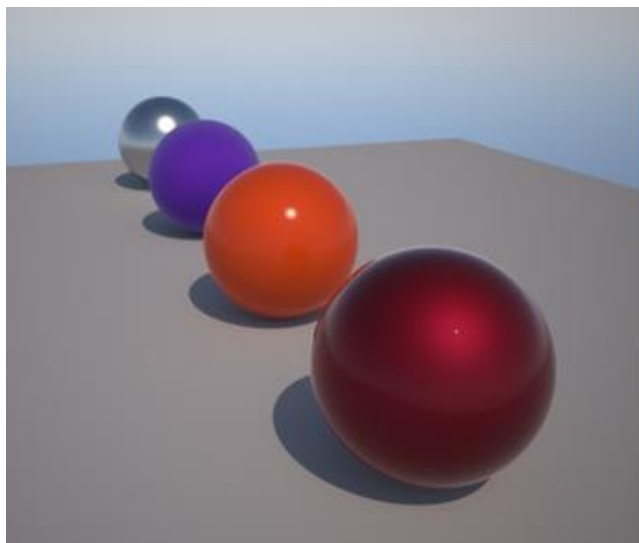
```
ZBuffer(fBuffer, zBuffer){  
  for y = 0 to ymax      // 初始化  
    for x = 0 to xmax{    //  
      fBuffer(x,y) = BACKGROUND_VALUE; // write color  
      zBuffer(x,y) = 1;   // write BG depth to z-buffer  
    }  
  for each polygon do    // 遍历多边形  
    for each pixel in polygon's projection do{  
      pz = polygon's z-value at pixel coords(x,y);  
      if (pz < readZ(x,y) {  
        zBuffer(x,y) = pz;  
        fBuffer(x,y) = polygon's color;  
      }  
    }  
  }  
}
```

How to
do this

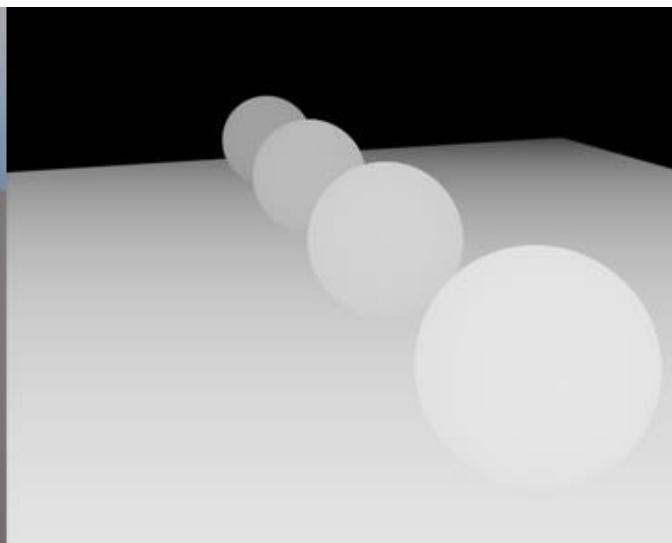
How to
do this

颜色与深度缓冲示意

颜色缓冲



深度缓冲



OpenGL中的消隐函数

- 1 为窗口选择深度缓存

```
glutInitDisplayMode (GLUT_DEPTH | .... );
```

- 2 激活深度测试

```
glEnable(GL_DEPTH_TEST);
```

- 3 设置深度/颜色缓存初始值

```
glClearDepth(1.0)/glClearColor(0,0,0,1);
```

- 4 执行清除

```
glClear(GL_COLOR_BUFFER_BIT | //  
GL_DEPTH_BUFFER_BIT);
```

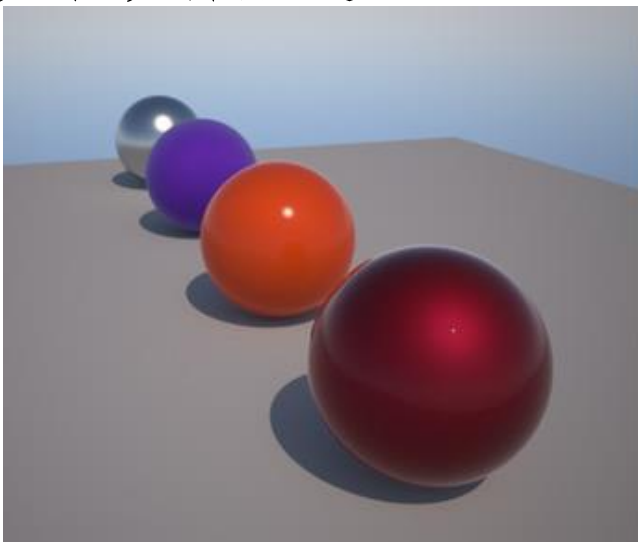
```
////////////////////////////////////
```

```
draw_3d_object_A(); draw_3d_object_B();
```

优缺点(Pros and cons)

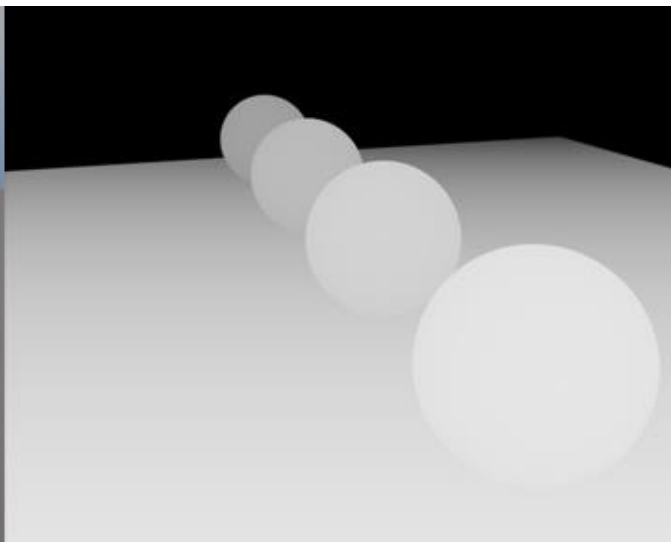
● 优点

- 不需要排序(Determine **Maximum** / **minimum**)
- 可处理任意几何形状
- 便于硬件加速



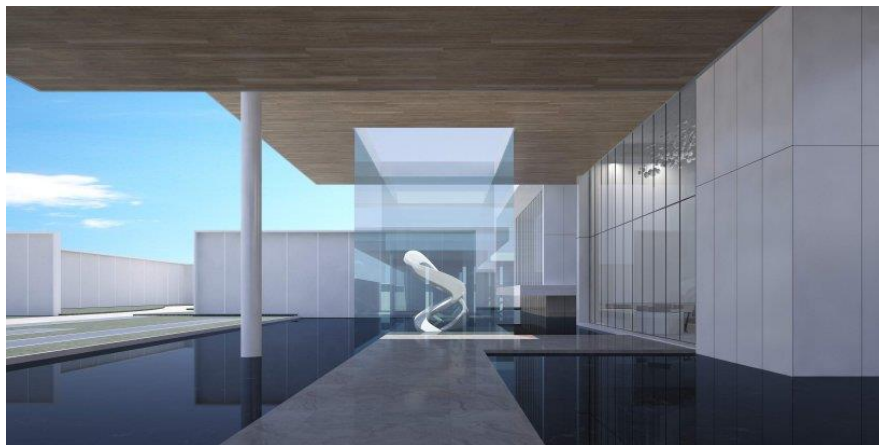
● 缺点:

- 复杂度高: ($O(nN)$)
- 消耗显存



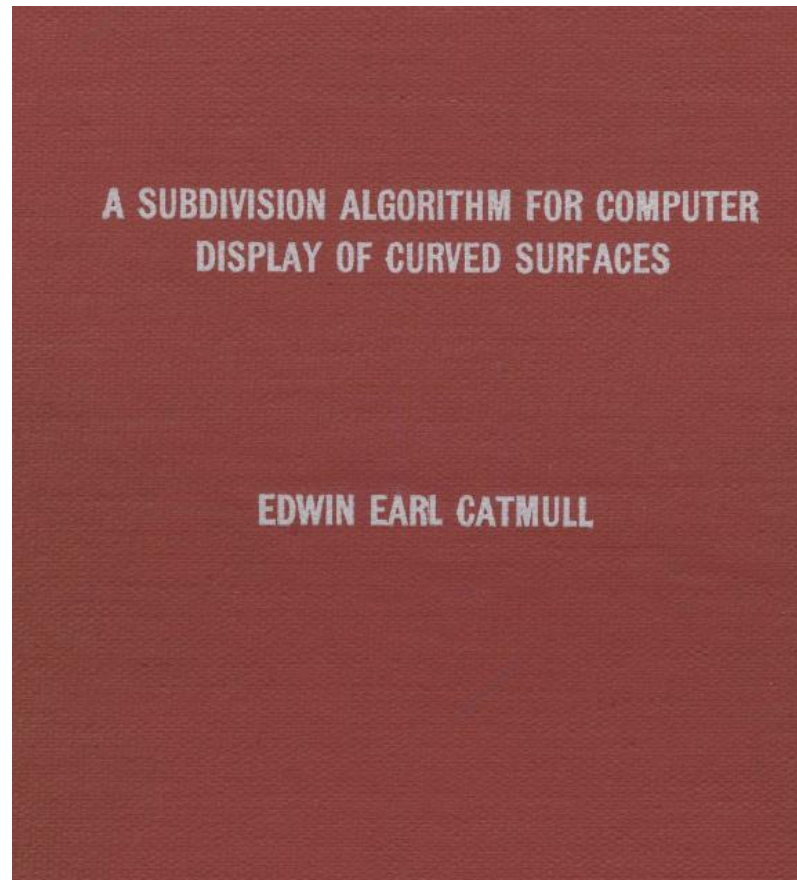
Z-buffer算法的优缺点

- 占大量显存 *Z-buffer/F-buffer*
 - 深度值范围从0到 10^6 ，则每像素 24 bits
 - 颜色缓存分辨率 1280×1024 ，则z-buffer需4MB现存
- 走样(Aliasing due to depth sampling)
- 难以处理透明物体
 - 颜色混合



Z-buffer reference

- E. E. Catmull. A Subdivision Algorithm for Computer Display of Curved Surfaces. Dissertation, University of Utah, 1974.



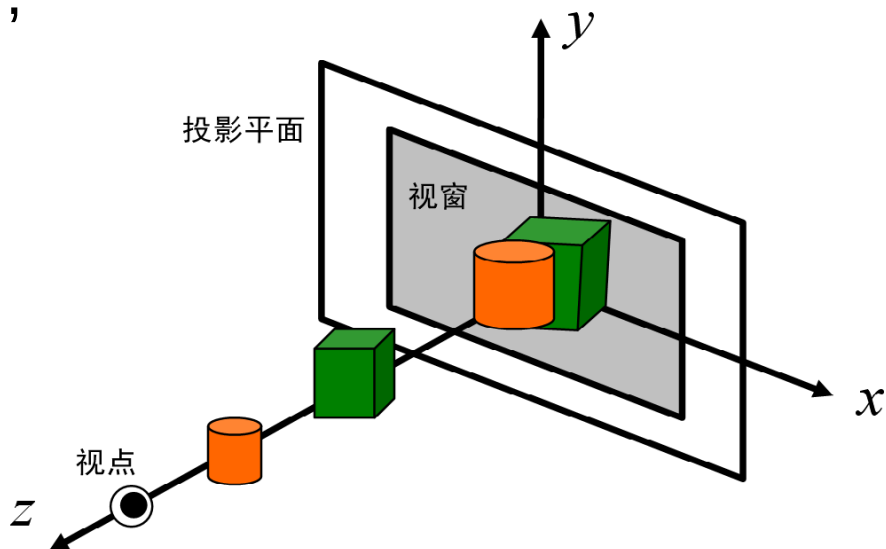
Agenda

- 消隐
 - 基本概念
 - 图像空间消隐
 - Z-buffer算法
 - 对象空间消隐
 - 后向相面剔除
 - 表优先级算法

3 对象空间消隐--算法框架

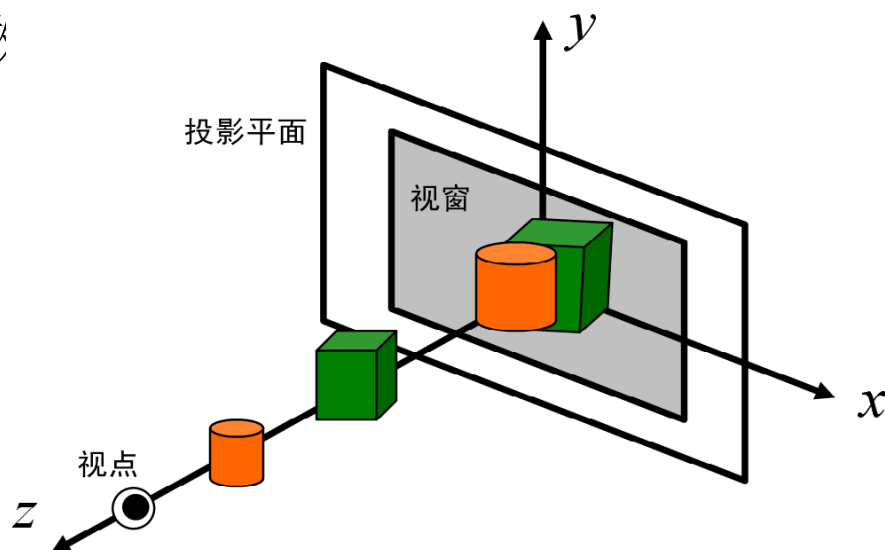
(HSR in object space)

1. for(场景中的每个物体(多边形)) {
2. 确定其未被其他物体遮挡的部分;
3. 进行扫描转换求出其投影包含的像素;
4. 计算像素的颜色;
5. }



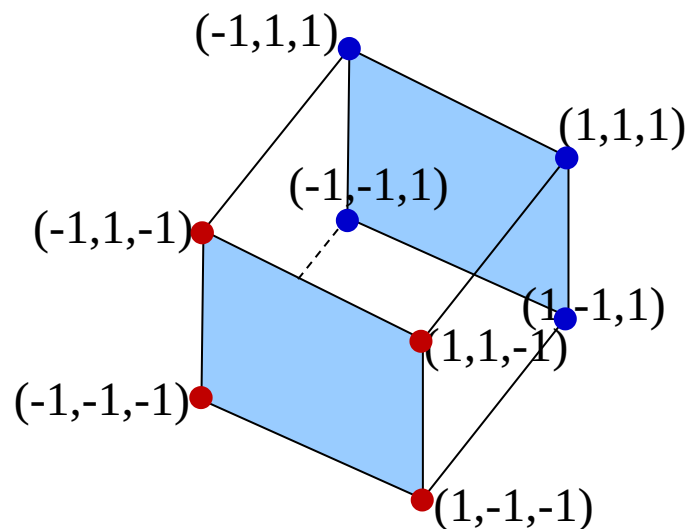
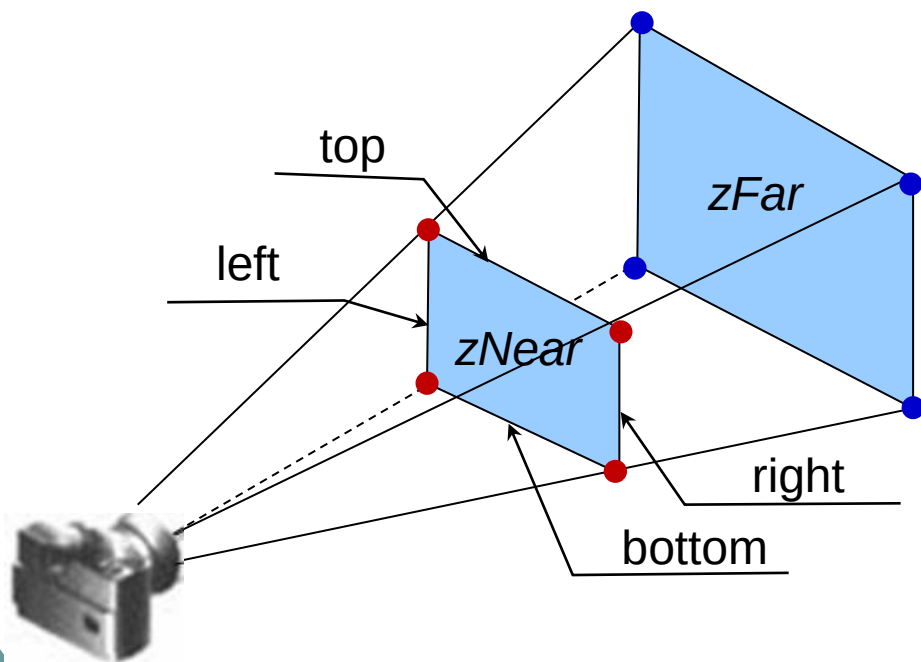
对象空间算法特点

- 适合于精密的CAD工程领域
 - 与渲染图像的分辨率无关
- 复杂度
 - $O(n^2)$, n : 多边形个数
- 后向面剔除
(Back surface culling)



消隐假设

- 场景由多边形面构成
- 离视点越远， z 值越大
- 在规范化立方体中消隐



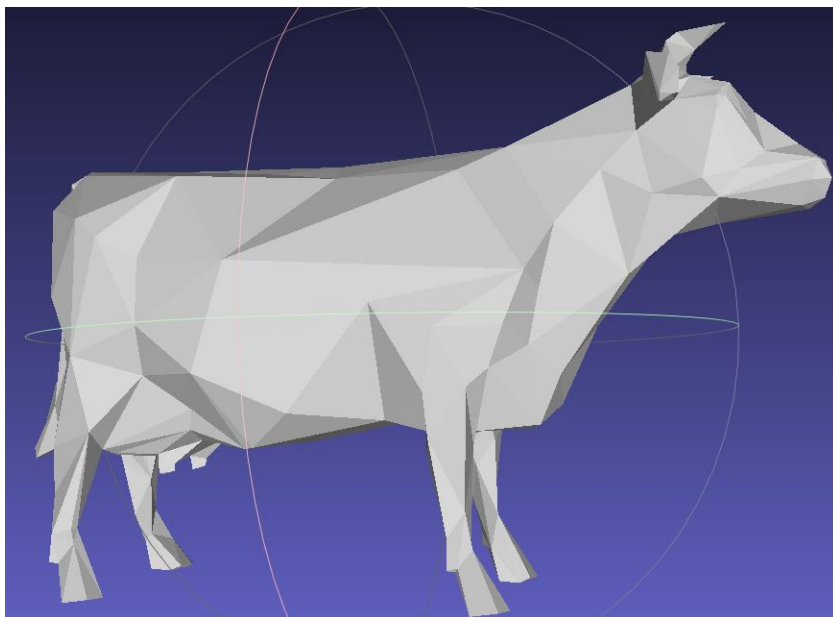
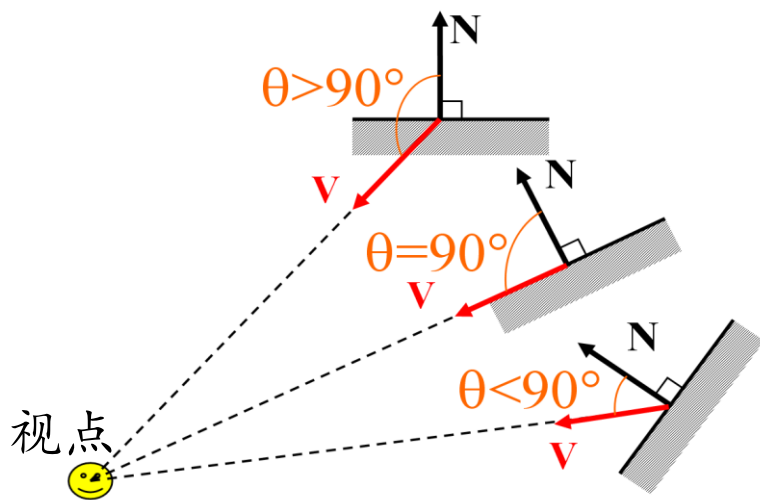
3. 1 后向面剔除(Back face culling)

● 测试场景中的每个多边形

- V是从多边形任一点到视点的方向；
- N是被测试多边形外法向

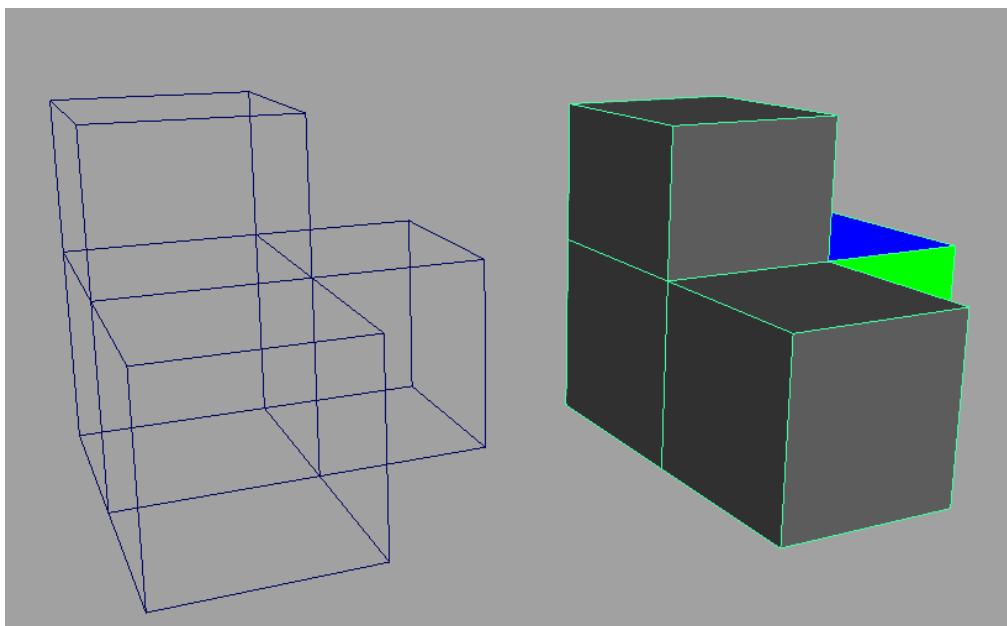
$N \cdot V < 0$: 不可见

$N \cdot V \geq 0$: 可见



后向面剔除的局限性

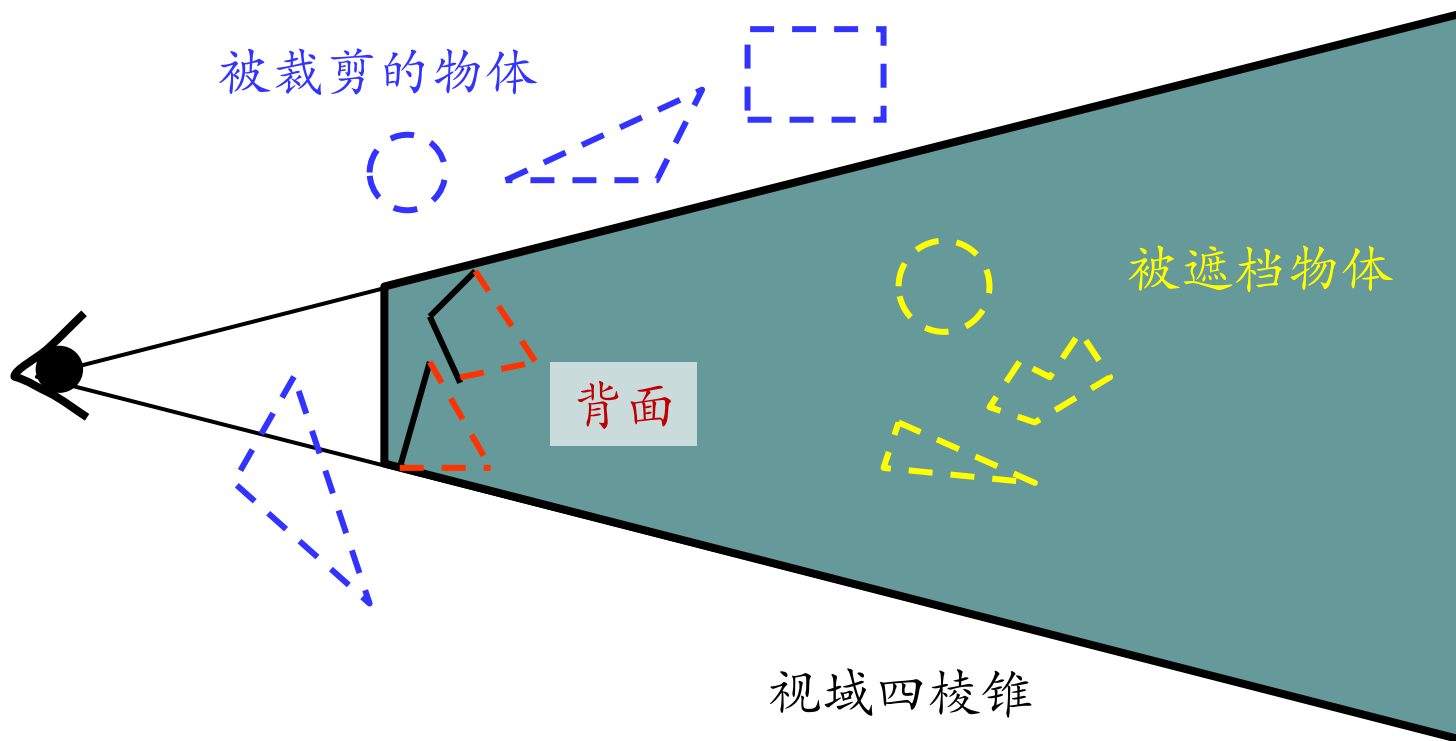
- 不能处理遮挡问题
 - 对凸多面体场景才能完全消隐



图中蓝色与绿色的面，简单的背面剔除不能实现完全消隐

后向面剔除的局限性

- 不能处理遮挡
 - 可以当作图像/对象两种消隐的预处理



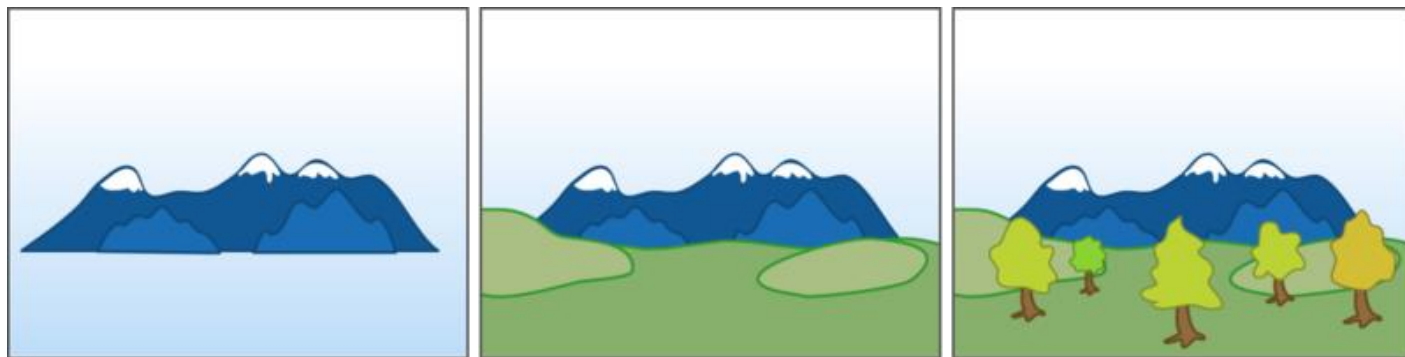
Agenda

- 隐藏面消除(HSR)
 - 基本概念(Terminologies)
 - 图像空间算法(Image space)
 - Z-buffer算法(深度缓存)
 - 对象空间算法(Object space)
 - 后向面剔除 (back surface culling)
 - 表优先级算法(list priority methods)
 - 深度排序算法/画家算法/
 - 二叉划分树算法

3.2 深度排序/表优先级/画家算法

● 原理

- 离视点远的不会遮挡离视点近的
- 在景物空间中确定物体间的可见性顺序
- 由远及近地绘制不会破坏遮挡关系——**油画家算法**



● 条件

- 场景中物体在 z 方向上没有相互重叠

深度排序算法的任务

- 确定场景中的多边形之间的相互遮挡关系



深度排序算法—z坐标不重叠多边形比较

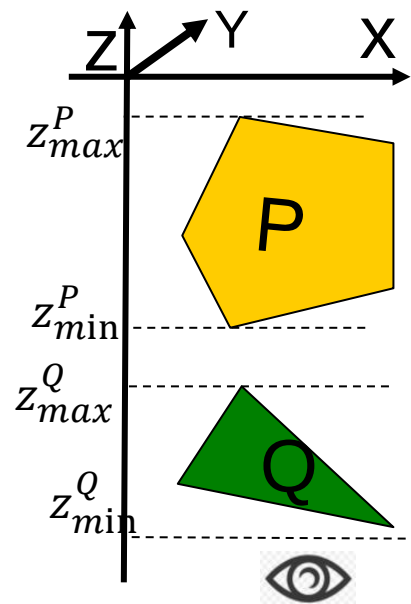
● 记 m 边形 $P = \{(x_1^P, y_1^P, z_1^P), \dots, (x_m^P, y_m^P, z_m^P)\}$

- $z_{max}^P = \max\{z_1^P, \dots, z_m^P\}$, $z_{min}^P = \min\{z_1^P, \dots, z_m^P\}$
- $x_{max}^P = \max\{x_1^P, \dots, x_m^P\}$, $x_{min}^P = \min\{x_1^P, \dots, x_m^P\}$
- $y_{max}^P = \max\{y_1^P, \dots, y_m^P\}$, $y_{min}^P = \min\{y_1^P, \dots, y_m^P\}$

1. 多边形 P 和 Q 的 z 坐标不重叠:

$$z_{max}^Q \leq z_{min}^P$$

P 排在 Q 的前面, 先绘制 P 后绘制 Q



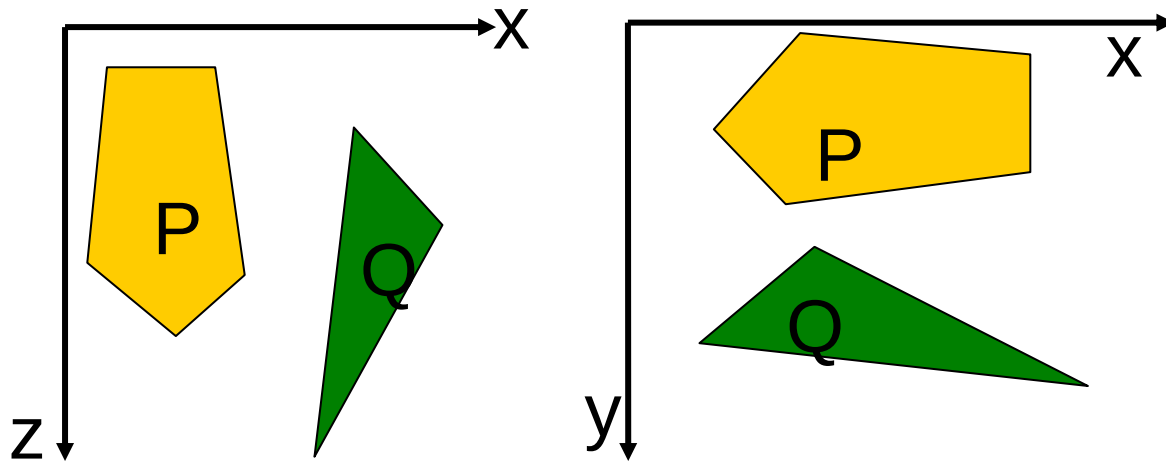
深度排序算法— x, y 坐标不重叠

2. z 值范围重叠时, 检查 P 和 Q 的 x, y 坐标区间是否重叠

(1) P 和 Q 的 x 坐标范围不重叠: $x_{max}^P \leq x_{min}^Q$

(2) P 和 Q 的 y 坐标范围不重叠: $y_{max}^P \leq y_{min}^Q$

两种情形都互不遮挡, 将 P 排在 Q 的前面, 先绘制 P 后 Q



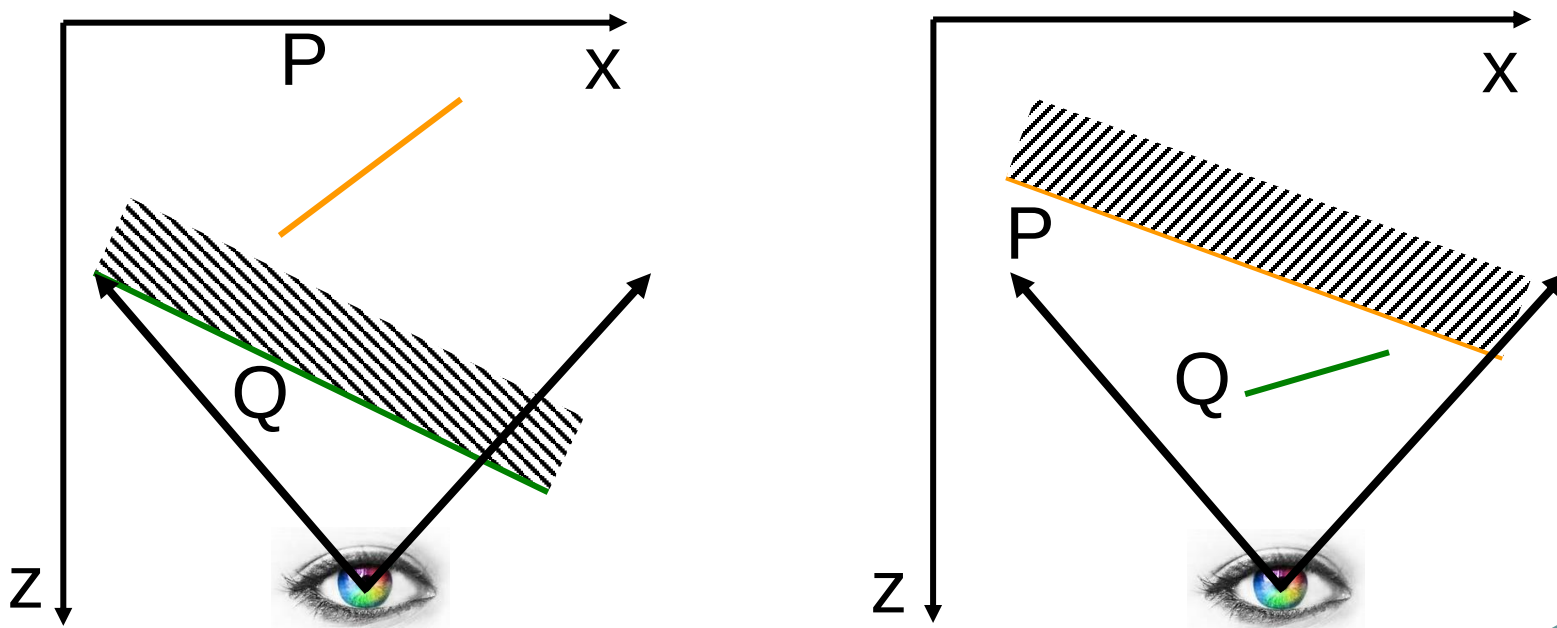
深度排序算法—是否与视点同侧

3. 如果1,2都无法确定 P, Q 顺序, 检查它们与视点关系

(1) 从视点看去, P 是否完全位于 Q 的背面

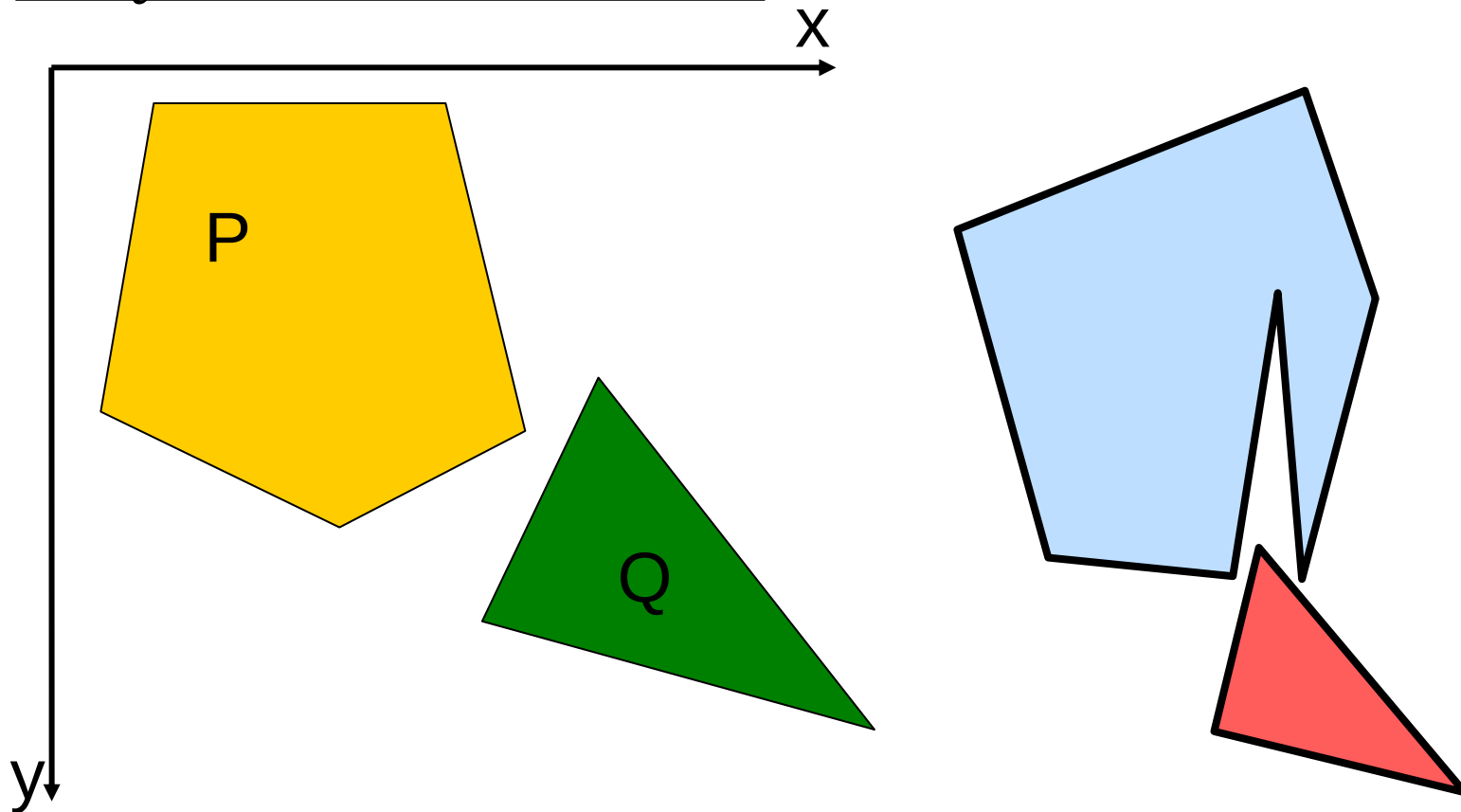
(2) 否则, 从视点看去, Q 是否完全位于 P 的同一侧

上面有一种情况成立, P 排在前, Q 在后, 先渲染 P , 后渲染 Q



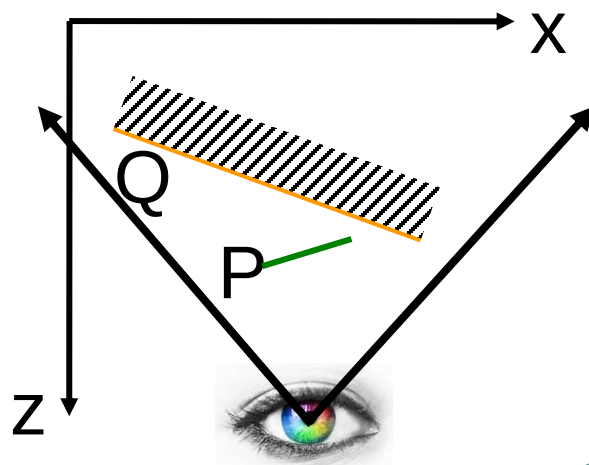
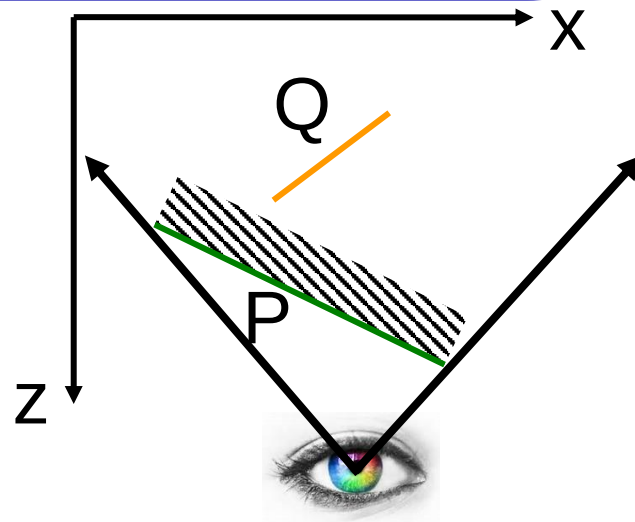
深度排序算法—XOY投影是否与重叠

4. P 和 Q 在平面上的投影不重叠 (如何判断?)



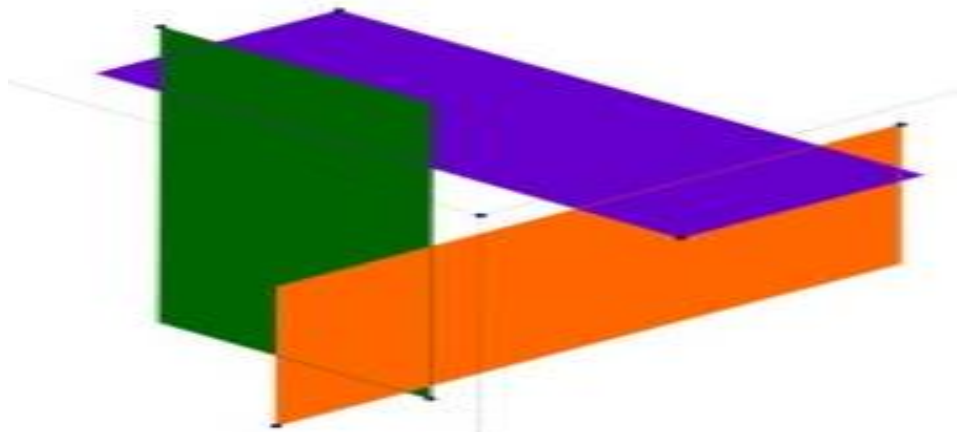
深度排序算法—交换P,Q顺序

- 前面4种情况有一种成立
 - 说明多边形 P 不会遮挡 Q
 - 即多边形 P 的绘制优先级高于 Q ，应先绘制
- 4种情况都不成立
 - P 有可能遮挡 Q
 - 互换 P 和 Q ，然后重新判断条件1, 3；即反过来考虑 Q 不会遮挡 P 的测试



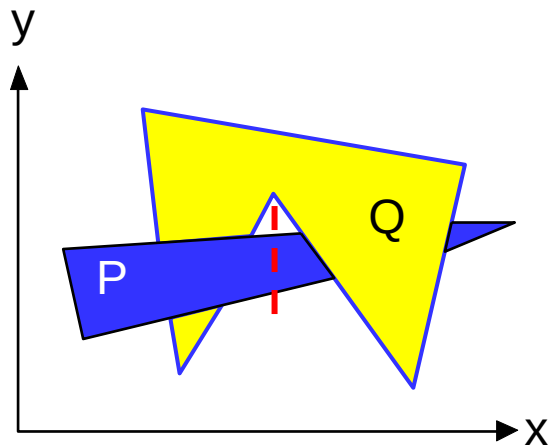
深度排序算法—求交

5. P, Q 交换顺序后，仍不能判断遮挡顺序
- 将其中一个多边形沿另一个物体剖分
 - 避免循环判断：P做标记
 - 多边形剖分：将P沿Q剖分



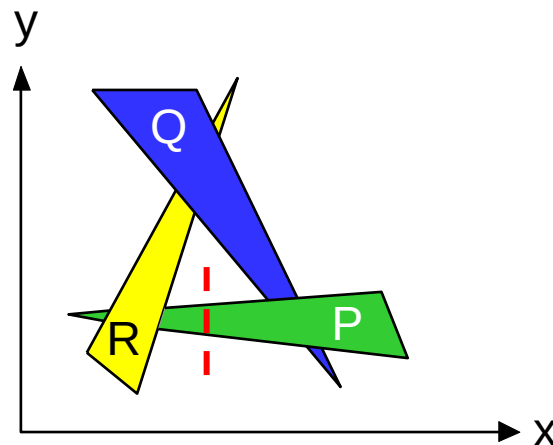
深度排序算法—求交的退化情形

相互遮挡时，将其中一个多边形沿另一个多边形进行剖分



$$P < Q, R < P, Q < R$$

循环遮挡，选中一个多边形进行裁剪。



深度排序算法—小结

- 深度排序**适合固定视点**消隐
 - 通过多边形剖分，总可以实现多边形物体在三维空间中的深度排序
 - 深度排序算法可以有效地实现透明效果
- 对视点变化的场合**(如飞行模拟)**，深度排序难以满足实时性要求
 - 算法复杂度 $O(n\log n)$



Painter algorithm-- Newell, Newell, Sancha

- M. E. Newell, R. G. Newell, T. L. Sancha. A new approach to the shaded picture problem. Proceedings of the ACM, 1973.

图形小史——那盏嘭瑟的茶壶 - 知乎 (zhihu.com)

内容

- 消隐的基本概念
- 图像空间消隐：z缓冲器(*z-buffer*)算法
- 物体空间消隐
 - 背面剔除算法
 - 表优先级算法
 - 三维物体的深度排序算法
 - 二叉空间剖分树算法

3.3 二叉空间剖分树

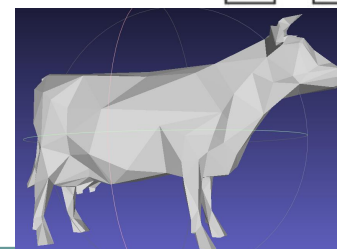
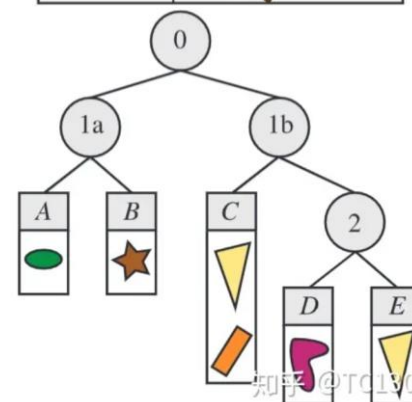
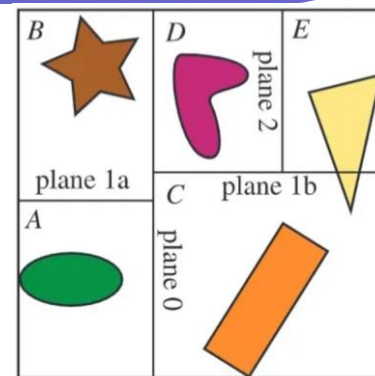
(Binary Space Partitioning, BSP)

● 二叉空间剖分树消隐算法

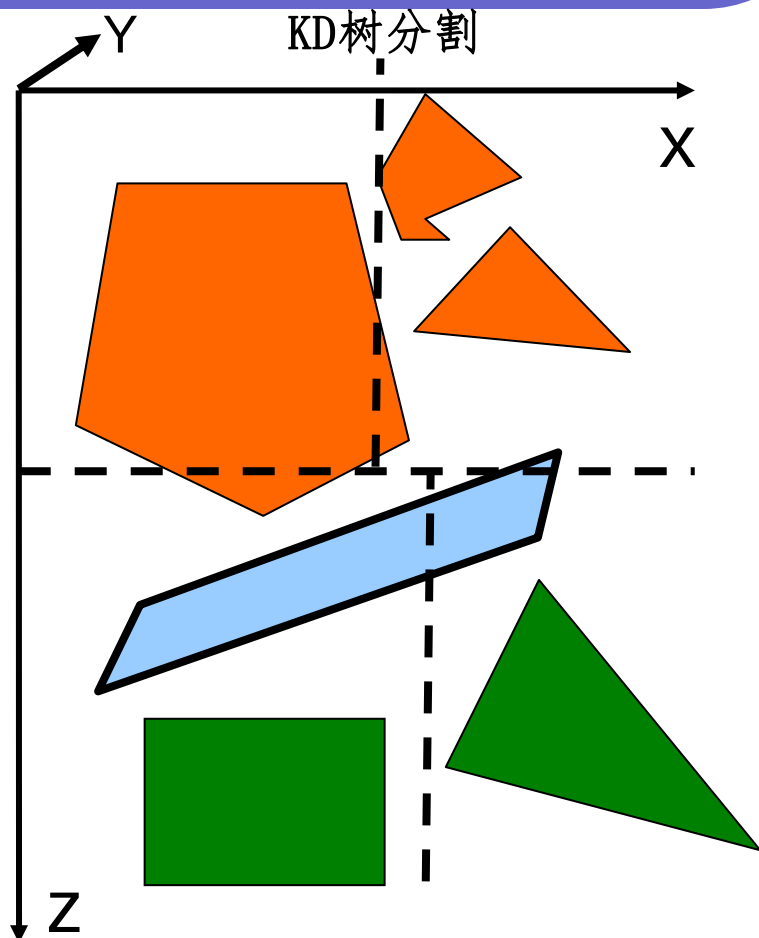
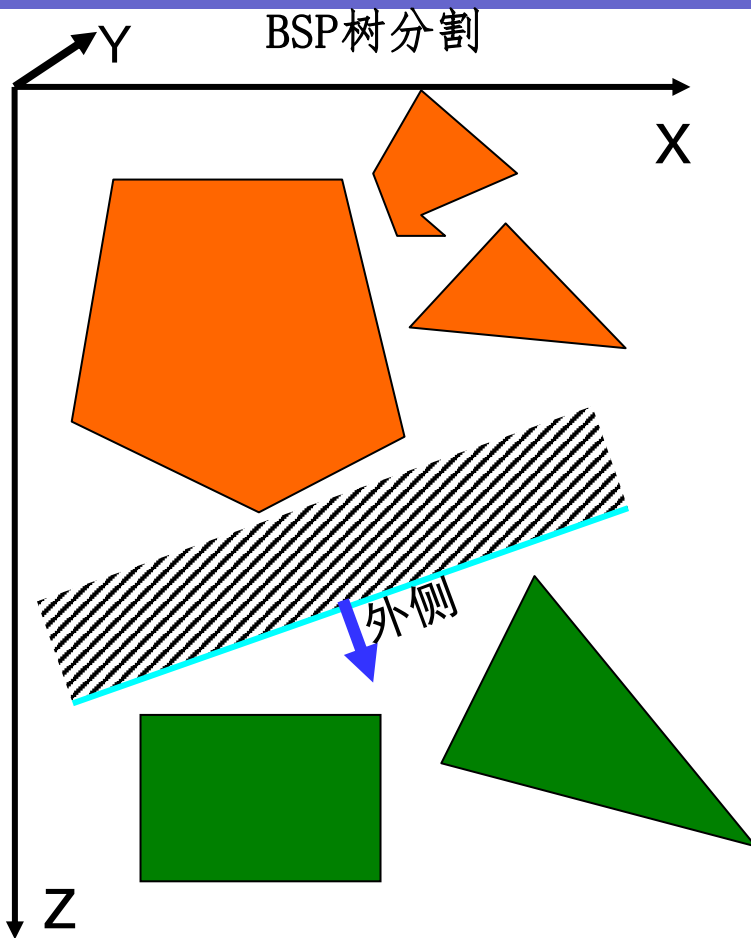
- 将场景中的多边形组织成一棵二叉树
- 对象空间算法，观察变换之前创建
- 适用于视点变化场景不变的绘制；

● 算法流程

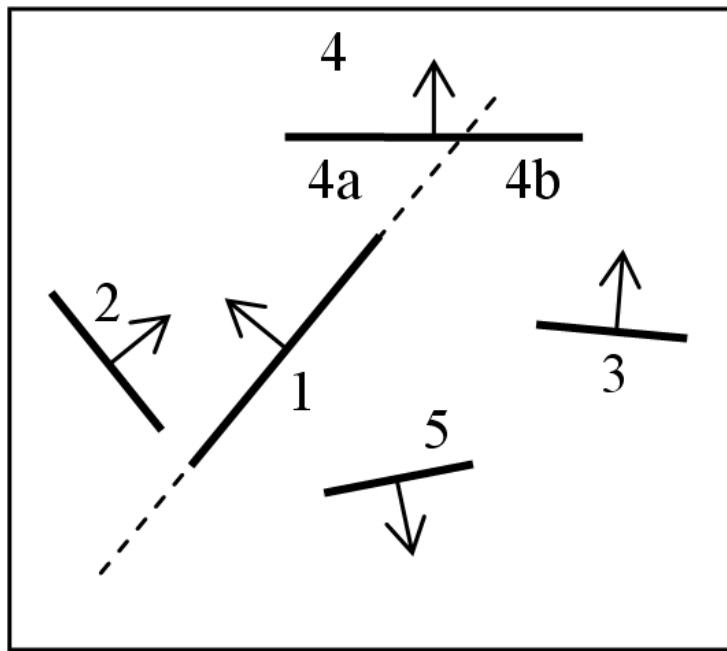
- 随机选场景中多边形P(也可选跟坐标系平行的平面，此时叫KD树)
- 将其它多边形按视点位置分成两部分
 - 位于P外侧(朝向模型外部)的多边形集合
 - 位于P内侧(朝向模型内部)的多边形集合
- 递归地建立两个子集合的子树



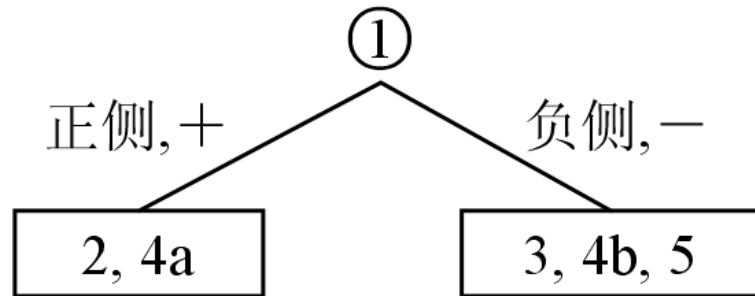
二叉空间剖分树--两种类型的分割面



二叉空间剖分树—树的生成

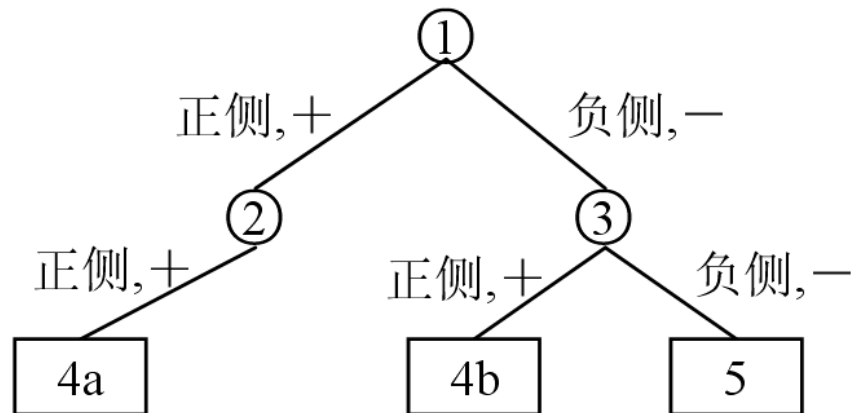
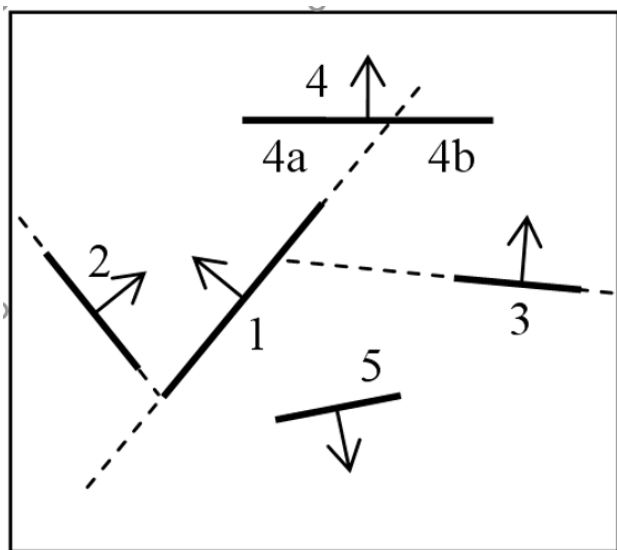


(a)



箭头表示多边形的正侧。左图：首先选1作为分割平面，2位于1的正侧，3和5位于1的负侧；4被1分割为4a和4b，其中4a位于1的正侧，4b位于1的负侧。右图：二叉树。

二叉空间剖分树—树的生成

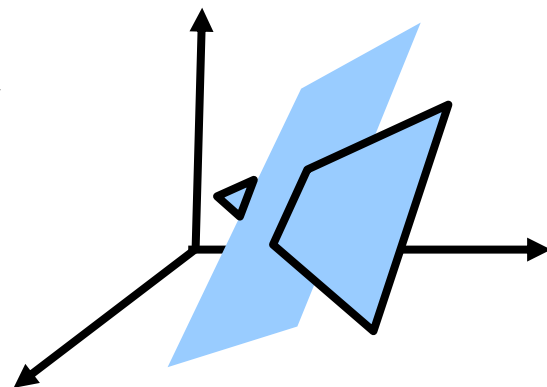


对左右两棵子树的进一步分割：

- 左子树：取2所在平面为分割平面，4a位于2的正侧；
 - 右子树：取3所在平面为分割平面，4b位于3的正侧，5位于3的负侧
- 至此建立了对所给场景的BSP树，树的每个叶节点也是一个多边形。

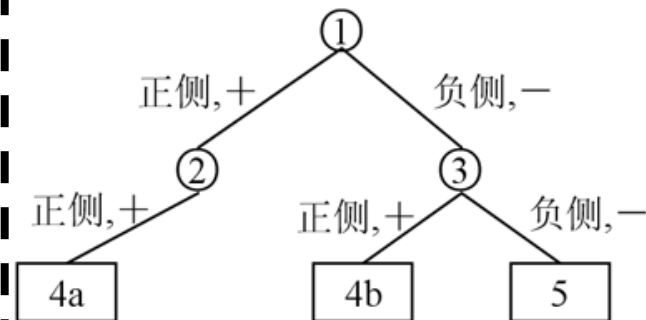
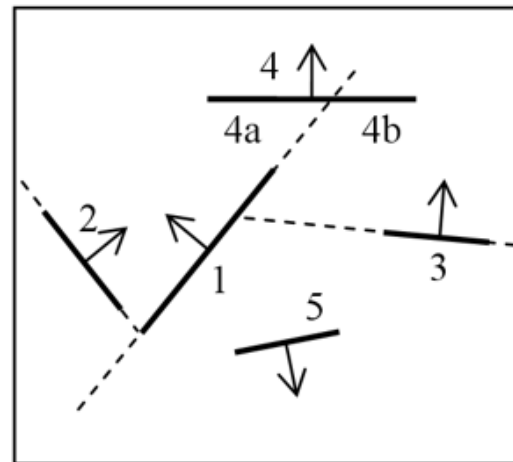
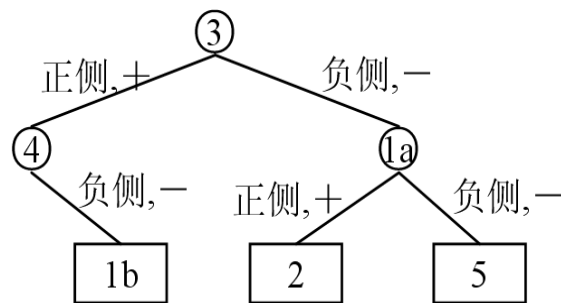
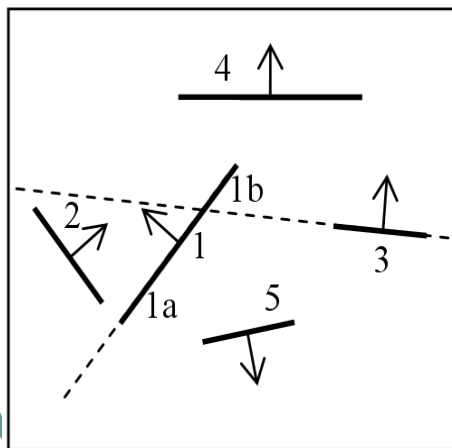
二叉空间剖分树—多边形分割

- 设P为选定多边形，其三维空间平面方程
$$f(x, y, z) = ax + by + cz + d = 0,$$
且满足：(x, y, z)位于外侧，则 $p(x, y, z) > 0$
- $Q = \{(x_1^Q, y_1^Q, z_1^Q), \dots, (x_n^Q, y_n^Q, z_n^Q)\}$ 是n边形
 - $f(x_i^Q, y_i^Q, z_i^Q) \geq 0, i = 1, \dots, n$, 外侧
 - $f(x_i^Q, y_i^Q, z_i^Q) \leq 0, i = 1, \dots, n$, 内侧
 - 上两个条件都不满足
 - 用平面 $f(x, y, z)$ 对Q进行多边形裁剪



二叉空间剖分树—唯一性问题

- 场景的BSP树不唯一
- “最佳” BSP树的两个标准
 - BSP树尽可能平衡
 - 尽可能减少多边形的剖分



二叉空间剖分树生成—伪代码

```
BSP_Tree BSP_MakeTree(PolygonList)
if PolygonList == NULL then BSPTree=NULL;
else {
    PartitionPolygon = SelectAndRemove(PolygonList);
    PositiveBranch = NegativeBranch = NULL;
    for ( each polygon P in PolygonList) {
        if( P in the positive side of PartitionPolygon)
            AddPolygonToBSP(P, PositiveBranch);
        else if (P in the negative side of PartitionPolygon )
            AddPolygonToBSP(P, NegativeBranch);
        else {
            SubdividePolygon(P, PartitionPolygon, PosiP, NegaP);
            AddPolygonToBSP(PosiP, PositiveBranch);
            AddPolygonToBSP(NegaP, NegativeBranch);
        }
    }
    CombineBSPTree(PositiveBranch, PartitionPolygon, NegativeBranch);
}
```

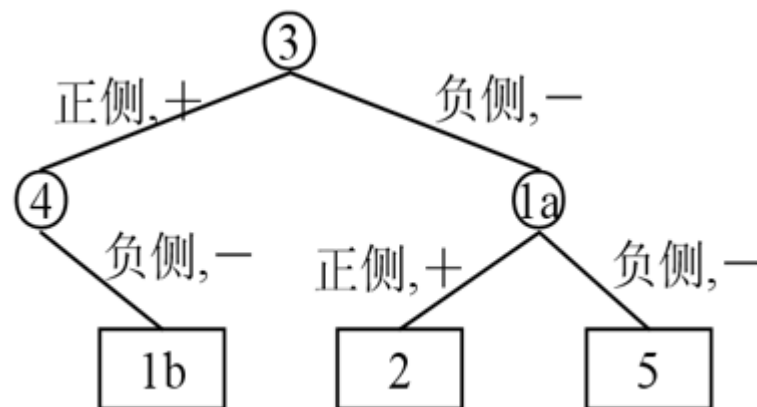
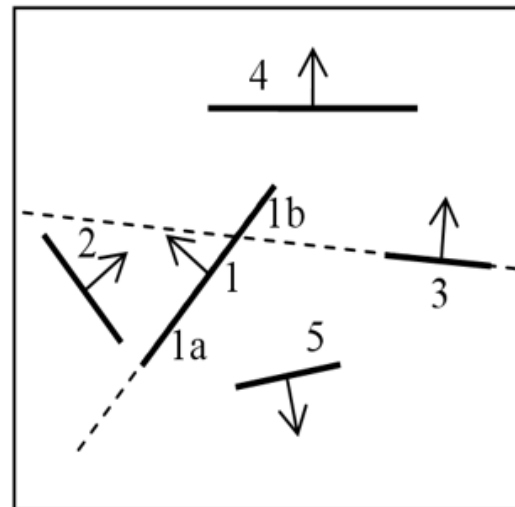
二叉空间剖分树--遍历(场景渲染)

- 前序遍历(已绘制像素不再绘制)

- 后序遍历(由远到近渲染)

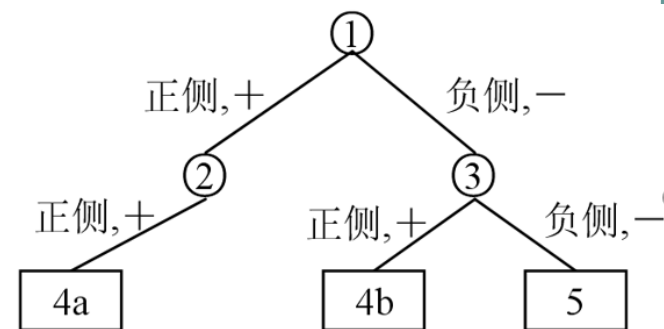
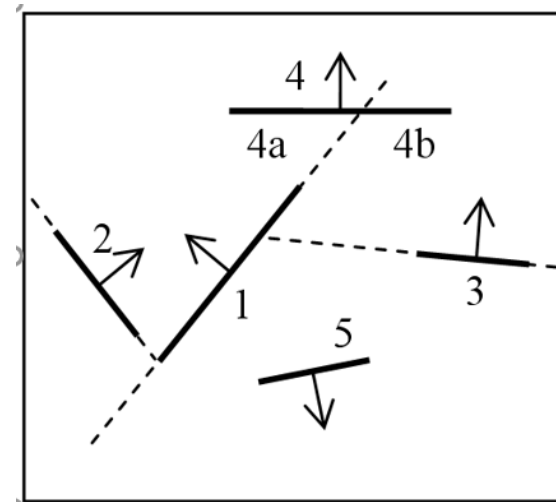
- 视点位于分割平面的正侧，遍历顺序：
负侧分支→根多边形→正侧分支

- 视点位于分割平面的负侧，遍历顺序：
正侧分支→根多边形→
负侧分支



二叉空间剖分树的遍历--伪代码

```
void showBSP(v: Viewer, T: BSPtree) {  
    if (T is empty) return;  
    P = root of T;  
    if (viewer is in front of P) {  
        showBSP(back subtree of T);  
        draw P;  
        showBSP(front subtree of T);  
    } else {  
        showBSP(front subtree of T);  
        draw P;  
        showBSP(back subtree of T);  
    }  
}
```



Henry Fuchs

- Federico Gil Professor of Computer Science, at the University of North Carolina at Chapel Hill.
- He has been active in computer graphics since the early 1970s, with rendering algorithms

(BSP Trees), hardware (Pixel-Planes and PixelFlow), virtual environments, tele-immersion systems and medical applications.



- He received a Ph.D. in 1975 from the University of Utah. He is a member of the National Academy of Engineering, a fellow of the American Academy of Arts and Sciences; the recipient of the **1992 ACM SIGGRAPH Achievement Award**, and the 1992 Academic Award of the National Computer Graphics Association.

小结

- 消隐的基本概念
- 图像空间消隐：z缓冲器(*z-buffer*)算法
- 物体空间消隐
 - 背面剔除算法
 - 表优先级算法
 - 三维物体的深度排序算法
 - 二叉空间剖分树算法